Università degli Studi di Napoli Federico II

Corso di Laurea in Informatica

RELAZIONE TECNICA DI PROGETTO

Modulo: Programmazione Object Oriented

Sistema Gestionale

"Concessionario Auto & Moto"

Gruppo di Lavoro:

Ronca Francesco - N86005712
Narretti Claudio J. - N86005495

Anno accademico 2024-2025

Indice

1 Introduzione **3**

2 Descrizione del Dominio **3**
2.1 Entità Principali . . . 3
2.2 Operazioni Principali . . . 3

3 Requisiti Funzionali e Non-Funzionali **4**
3.1 Requisiti Funzionali . . . 4
3.2 Requisiti Non-Funzionali . . . 4

4 Schema del Database Relazionale **5**
4.1 Tabelle Principali . . . 5
4.1.1 Tabella VEICOLI . . . 5
4.1.2 Tabella CLIENTI . . . 5
4.1.3 Tabella VENDITE . . . 5
4.1.4 Tabella TEST_DRIVE . . . 5

5 Regole di Business e Vincoli Semantici **6**
5.1 Implementazione dei Vincoli . . . 6

6 Architettura del Sistema **7**
6.1 Struttura Generale . . . 7
6.2 Pattern di Design Implementati . . . 7
6.2.1 Data Access Object (DAO) Pattern . . . 7
6.2.2 Singleton Pattern . . . 7
6.2.3 Pattern BCE (Boundary-Control-Entity) . . . 8

7 Classi e Interfacce Principali **9**
7.1 Classi di Modello . . . 9
7.1.1 Classe Veicolo . . . 9
7.1.2 Classe Cliente . . . 9
7.2 Interfacce e Implementazioni DAO . . . 9
7.2.1 Interface VeicoloDAO e VeicoloDAOImpl . . . 9
7.2.2 Interface ClienteDAO e ClienteDAOImpl . . . 10
7.2.3 Interface VenditaDAO e VenditaDAOImpl . . . 10
7.2.4 Interface TestDriveDAO e TestDriveDAOImpl . . . 11
7.3 Classe DatabaseManager . . . 11

8 Gerarchia delle Eccezioni **12**

9 Operazioni Critiche e Pseudocodice **13**
9.1 Operazione: Effettuazione Vendita . . . 13
9.2 Operazione: Richiesta Test Drive . . . 14

10 Scelte Progettuali e Principi SOLID **15**
10.1 Pattern Singleton per DatabaseManager . . . 15
10.2 Interfacce DAO per l'Astrazione . . . 15
10.3 Eccezioni Personalizzate . . . 15

10.4 Principi SOLID Applicati 15

11 Interfaccia Utente Grafica **16**
11.1 Classe MainApp 16
11.2 Viste Disponibili 16

12 Diagrammi UML **17**
12.1 Diagramma delle Classi 17
12.2 Diagramma del Pattern DAO 17
12.3 Diagramma dei Package 18
12.4 Diagramma dell'Architettura 19

13 Tecnologie Utilizzate **20**
13.1 Parametri di Connessione 20

14 Conclusioni **20**

1 Introduzione

Il progetto "Concessionario Auto & Moto" è un'applicazione desktop sviluppata in Java che implementa un sistema gestionale completo per la gestione di una concessionaria di veicoli.

L'applicazione fornisce funzionalità di base per la gestione di clienti, veicoli e transazioni di vendita, nonché la possibilità di richiedere test drive.

L'applicazione è stata sviluppata seguendo i paradigmi della Programmazione Object Oriented (POO) e implementa pattern di design riconosciuti, in particolare il pattern Data Access Object (DAO) per l'accesso ai dati. L'architettura è stratificata in modo da separare la logica di presentazione dalla logica di accesso ai dati.

2 Descrizione del Dominio

2.1 Entità Principali

Il sistema gestisce le seguenti entità principali:

- **Veicolo**: Rappresenta un veicolo (auto o moto) del concessionario con proprietà quali targa, marca, modello, tipo, prezzo e stato attuale della disponibilità.
- **Cliente**: Rappresenta un cliente del concessionario, che può essere un privato o un'azienda, con informazioni di contatto e un identificativo univoco.
- **Vendita**: Rappresenta la transazione di vendita di un veicolo a un cliente.
- **Test Drive**: Rappresenta la richiesta di un cliente di eseguire una prova di guida su un veicolo.

2.2 Operazioni Principali

Le operazioni principali supportate dal sistema sono:

1. Registrazione di nuovi clienti
2. Aggiunta di nuovi veicoli al catalogo
3. Visualizzazione dei veicoli disponibili
4. Modifica dei dati dei veicoli
5. Eliminazione di veicoli dal catalogo
6. Modifica dei dati dei clienti
7. Eliminazione di clienti
8. Effettuazione di vendite
9. Richiesta di test drive
10. Visualizzazione della lista degli ordini/transazioni

3 Requisiti Funzionali e Non-Funzionali

3.1 Requisiti Funzionali

Il sistema deve supportare i seguenti casi d'uso:

1. **Gestione Clienti**: Registrazione, modifica, eliminazione e visualizzazione di clienti (privato o azienda)
2. **Gestione Veicoli**: Aggiunta, modifica, eliminazione e visualizzazione di veicoli disponibili (auto o moto)
3. **Effettuazione Vendita**: Registrazione della transazione di vendita di un veicolo a un cliente con aggiornamento dello stato
4. **Richiesta Test Drive**: Registrazione della richiesta di un cliente per provare un veicolo
5. **Visualizzazione Ordini**: Consultazione della lista storica di vendite e test drive effettuati

3.2 Requisiti Non-Funzionali

- **Affidabilità**: L'applicazione deve gestire correttamente gli errori di connessione al database mediante eccezioni personalizzate
- **Usabilità**: Interfaccia intuitiva sviluppata con JavaFX per un'esperienza utente fluida e coerente
- **Manutenibilità**: Codice organizzato secondo pattern di design riconosciuti (DAO, Singleton, BCE) e principi SOLID
- **Integrità dei Dati**: Validazione lato applicazione e vincoli a livello database per garantire la consistenza dei dati
- **Persistenza**: Utilizzo di PostgreSQL per la memorizzazione persistente e duratura dei dati

4 Schema del Database Relazionale

4.1 Tabelle Principali

Il database è composto dalle seguenti tabelle:

4.1.1 Tabella VEICOLI

- **targa** (VARCHAR, PK): Identificativo univoco del veicolo
- **marca** (VARCHAR, NOT NULL): Casa produttrice
- **modello** (VARCHAR, NOT NULL): Modello del veicolo
- **tipo** (VARCHAR, NOT NULL, CHECK IN ('auto', 'moto')): Tipo di veicolo
- **prezzo** (NUMERIC, NOT NULL, CHECK ¿ 0): Prezzo del veicolo
- **stato** (VARCHAR, DEFAULT 'disponibile'): Stato attuale (disponibile, venduto, in_test_drive)

4.1.2 Tabella CLIENTI

- **id** (SERIAL, PK): Identificativo univoco del cliente
- **nome** (VARCHAR, NOT NULL): Nome del cliente
- **tipo** (VARCHAR, NOT NULL, CHECK IN ('privato', 'azienda')): Tipo di cliente
- **email** (VARCHAR, NOT NULL, UNIQUE): Indirizzo email
- **telefono** (VARCHAR, NOT NULL): Numero di telefono

4.1.3 Tabella VENDITE

- **id** (SERIAL, PK): ID della vendita
- **targa_veicolo** (VARCHAR, FK → VEICOLI): Riferimento al veicolo venduto
- **id_cliente** (INTEGER, FK → CLIENTI): Riferimento al cliente acquirente
- **data_vendita** (DATE, DEFAULT CURRENT_DATE): Data della transazione
- **prezzo_finale** (NUMERIC, NOT NULL): Prezzo della vendita

4.1.4 Tabella TEST_DRIVE

- **id** (SERIAL, PK): ID del test drive
- **id_cliente** (INTEGER, FK → CLIENTI): Riferimento al cliente
- **targa_veicolo** (VARCHAR, FK → VEICOLI): Riferimento al veicolo
- **data_richiesta** (DATE, DEFAULT CURRENT_DATE): Data della richiesta
- **stato** (VARCHAR, DEFAULT 'richiesto'): Stato del test drive (richiesto, confermato, completato)

5 Regole di Business e Vincoli Semantici

Il sistema implementa le seguenti regole di business:

1. **Univocità Targa**: Una targa non può comparire due volte nel catalogo dei veicoli (Constraint PRIMARY KEY)
2. **Tipi Veicolo**: Un veicolo deve essere classificato come 'auto' o 'moto' (Constraint CHECK)
3. **Tipi Cliente**: Un cliente deve essere classificato come 'privato' o 'azienda' (Constraint CHECK)
4. **Prezzo Positivo**: Il prezzo di un veicolo deve essere maggiore di zero (Constraint CHECK in DB e validazione in Java)
5. **Email Valida**: L'indirizzo email di un cliente deve contenere il simbolo '@' (Validazione in Java)
6. **Integrità Referenziale**: Una vendita deve fare riferimento a un cliente e a un veicolo esistenti (Foreign Keys)
7. **Aggiornamento Stato Veicolo**: Quando un veicolo viene venduto, il suo stato deve essere aggiornato a 'venduto' (Logica applicativa in VenditaDAOImpl)
8. **Impossibilità Doppia Vendita**: Un veicolo venduto non può essere venduto di nuovo (Validazione in EffettuaVenditaView)

5.1 Implementazione dei Vincoli

Regola di Business	**Implementazione**	**Ubicazione**
Univocità Targa	PRIMARY KEY	Database (schema.sql)
Tipi Veicolo	CHECK constraint	Database (schema.sql)
Tipi Cliente	CHECK constraint	Database (schema.sql)
Prezzo Positivo	CHECK constraint + Validazione Java	Database + Classe Veicolo
Email Valida	Validazione Java	Classe Cliente
Integrità Referenziale	Foreign Keys	Database (schema.sql)
Aggiornamento Stato	Logica applicativa	VenditaDAOImpl.effettuaVendita()
Impossibilità Doppia Vendita	Validazione	EffettuaVenditaView

6 Architettura del Sistema

6.1 Struttura Generale

L'applicazione è organizzata secondo un'architettura stratificata con la seguente struttura di package:

- **it.unina.prog.model**: Contiene le classi di modello (Cliente, Veicolo) che rappresentano le entità del dominio.
- **it.unina.prog.dao**: Contiene le interfacce DAO (Data Access Object) per l'accesso ai dati.
- **it.unina.prog.dao.impl**: Contiene le implementazioni concrete delle interfacce DAO.
- **it.unina.prog.db**: Contiene la classe DatabaseManager per la gestione della connessione al database.
- **it.unina.prog.gui**: Contiene le classi di vista per l'interfaccia utente grafica sviluppate con JavaFX.
- **it.unina.prog.exception**: Contiene le classi di eccezione personalizzate.
- **it.unina.prog.controller**: Contiene i controller per la gestione degli eventi.

6.2 Pattern di Design Implementati

6.2.1 Data Access Object (DAO) Pattern

Il pattern DAO è stato implementato per astrarre e incapsulare i dettagli dell'accesso ai dati dal resto dell'applicazione. Per ogni entità principale (Veicolo, Cliente, Vendita, TestDrive) è stata definita un'interfaccia DAO corrispondente:

- **VeicoloDAO**: Interfaccia per le operazioni sui veicoli
- **ClienteDAO**: Interfaccia per le operazioni sui clienti
- **VenditaDAO**: Interfaccia per le operazioni di vendita
- **TestDriveDAO**: Interfaccia per le operazioni di test drive

Le implementazioni concrete (VeicoloDAOImpl, ClienteDAOImpl, VenditaDAOImpl, TestDriveDAOImpl) implementano queste interfacce e gestiscono l'interazione con il database PostgreSQL attraverso le query SQL.

6.2.2 Singleton Pattern

Il pattern Singleton è stato implementato nella classe DatabaseManager per garantire che esista una sola istanza del gestore della connessione al database durante l'esecuzione dell'applicazione.

6.2.3 Pattern BCE (Boundary-Control-Entity)

L'applicazione implementa il pattern architetturale Boundary-Control-Entity (BCE) in combinazione con il pattern DAO:

- **Boundary**: Layer di presentazione implementato dalle classi in `it.unina.prog.gui` (MainApp, VisualizzaVeicoliView, RegistraClienteView, etc.). Gestisce l'interfaccia utente JavaFX e la comunicazione con l'utente.
- **Control**: Layer di logica applicativa che coordina le operazioni tra Boundary e Entity. Include eventuali controller e la logica di business.
- **Entity**: Layer di modello del dominio definito dalle classi in `it.unina.prog.model` (Cliente, Veicolo) che rappresentano le entità del sistema.
- **DAO (Data Access Object)**: Layer di persistenza (package `dao` e `dao.impl`) che incapsula completamente l'accesso ai dati, permettendo di sostituire facilmente il database.

Questa architettura garantisce una chiara separazione delle responsabilità e facilita la manutenibilità, testabilità ed estensibilità del sistema.

7 Classi e Interfacce Principali

7.1 Classi di Modello

7.1.1 Classe Veicolo

La classe Veicolo rappresenta un veicolo nel sistema. Gli attributi principali sono:

- **targa**: Identificativo univoco del veicolo (final, immutabile)
- **marca**: Marca del produttore
- **modello**: Modello del veicolo
- **tipo**: Tipo di veicolo (auto o moto)
- **prezzo**: Prezzo del veicolo
- **stato**: Stato attuale del veicolo (disponibile, venduto, in test drive)

La classe implementa validazione nei propri costruttori e setter per garantire l'integrità dei dati.

7.1.2 Classe Cliente

La classe Cliente rappresenta un cliente del concessionario. Gli attributi principali sono:

- **id**: Identificativo univoco del cliente (final, immutabile)
- **nome**: Nome completo del cliente
- **tipo**: Tipo di cliente (privato o azienda)
- **email**: Email del cliente
- **telefono**: Numero di telefono del cliente

Come per Veicolo, la classe implementa validazione nei propri costruttori e setter.

7.2 Interfacce e Implementazioni DAO

7.2.1 Interface VeicoloDAO e VeicoloDAOImpl

L'interfaccia VeicoloDAO definisce le operazioni disponibili per la gestione dei veicoli:

```java
public interface VeicoloDAO {
    void inserisciVeicolo(String targa, String marca, String modello,
                          String tipo, double prezzo) throws DatabaseException;

    String getStatoVeicolo(String targa) throws DatabaseException;

```

```
    List<Veicolo> getVeicoliDisponibili() throws DatabaseException;

    void aggiornaVeicolo(String targa, String marca, String modello,
                         String tipo, double prezzo) throws DatabaseException;

    void eliminaVeicolo(String targa) throws DatabaseException;

    List<Veicolo> getAllVeicoli() throws DatabaseException;
}
```

La classe VeicoloDAOImpl implementa questa interfaccia, fornendo l'implementazione concreta delle operazioni CRUD (Create, Read, Update, Delete) per i veicoli nel database PostgreSQL.

7.2.2 Interface ClienteDAO e ClienteDAOImpl

L'interfaccia ClienteDAO definisce le operazioni disponibili per la gestione dei clienti:

```
public interface ClienteDAO {
    void inserisciCliente(String nome, String tipo, String email,
                          String telefono) throws DatabaseException;

    List<Cliente> getClienti() throws DatabaseException;

    void aggiornaCliente(int id, String nome, String tipo, String email,
                         String telefono) throws DatabaseException;

    void eliminaCliente(int id) throws DatabaseException;

    Cliente getCliente(int id) throws DatabaseException;
}
```

La classe ClienteDAOImpl implementa questa interfaccia per le operazioni CRUD sui clienti.

7.2.3 Interface VenditaDAO e VenditaDAOImpl

L'interfaccia VenditaDAO definisce le operazioni per la registrazione delle vendite:

```
public interface VenditaDAO {
    void effettuaVendita(String targa, int clienteId, int dipendenteId,
                         double prezzoFinale) throws DatabaseException;
}
```

7.2.4 Interface TestDriveDAO e TestDriveDAOImpl

L'interfaccia TestDriveDAO definisce le operazioni per la gestione dei test drive:

```
public interface TestDriveDAO {
    void richiediTestDrive(int clienteId, String targa,
                           String data) throws DatabaseException;
}
```

7.3 Classe DatabaseManager

La classe DatabaseManager implementa il pattern Singleton e gestisce le connessioni al database PostgreSQL:

```
public class DatabaseManager {
    private static final String URL = "jdbc:postgresql://
   localhost:5432/concessionario_db";
    private static final String USER = "concessionario_admin";
    private static final String PASSWORD = "password123";

    private static DatabaseManager instance;

    private DatabaseManager() {}

    public static DatabaseManager getInstance() {
        if (instance == null) {
            instance = new DatabaseManager();
        }
        return instance;
    }

    public Connection getConnection() throws DatabaseException {
        try {
            return DriverManager.getConnection(URL, USER,
   PASSWORD);
        } catch (SQLException e) {
            throw new DatabaseException("Impossibile connettersi
   al database", e);
        }
    }
}
```

8 Gerarchia delle Eccezioni

L'applicazione definisce una gerarchia di eccezioni personalizzate per gestire in modo specifico i diversi tipi di errori:

- **ConcessionarioException**: Eccezione base del sistema
- **DatabaseException**: Eccezione per errori relativi all'accesso al database (estende ConcessionarioException)
- **ValidationException**: Eccezione per errori di validazione dei dati (estende ConcessionarioException)

9 Operazioni Critiche e Pseudocodice

9.1 Operazione: Effettuazione Vendita

Questa operazione è critica in quanto modifica lo stato del veicolo e crea un nuovo record di transazione.

```
// Precondizioni:
// - clienteId deve riferire a un cliente esistente
// - targa deve riferire a un veicolo esistente
// - veicolo.stato deve essere 'disponibile'
// - prezzoFinale deve essere > 0

Procedura effettuaVendita(targa, clienteId, prezzoFinale)
    INIZIO
        // 1. Validazione input
        SE targa e' vuota O clienteId <= 0 O prezzoFinale <= 0 ALLORA
            LANCIA ValidationException
        FINE SE

        // 2. Ottenere connessione al database
        Connection conn = DatabaseManager.getInstance().getConnection()

        // 3. Iniziare transazione
        conn.setAutoCommit(false)

        // 4. Verificare che il veicolo esista e sia disponibile
        String statoVeicolo = veicoloDAO.getStatoVeicolo(targa)
        SE statoVeicolo != 'disponibile' ALLORA
            LANCIA ValidationException("Veicolo non disponibile")
        FINE SE

        // 5. Verificare che il cliente esista
        Cliente cliente = clienteDAO.getCliente(clienteId)
        SE cliente == null ALLORA
            LANCIA ValidationException("Cliente non trovato")
        FINE SE

        // 6. Registrare la vendita nel database
        INSERT INTO vendite (targa_veicolo, id_cliente, data_vendita, prezzo_finale)
        VALUES (targa, clienteId, CURRENT_DATE, prezzoFinale)

        // 7. Aggiornare lo stato del veicolo a 'venduto'
        UPDATE veicoli SET stato = 'venduto' WHERE targa = targa

        // 8. Confermare la transazione
        conn.commit()

        RITORNA successo
```

```
    ECCEZIONE
        conn.rollback()  // Annulla tutte le modifiche
        LANCIA DatabaseException
    FINE ECCEZIONE
FINE
```

Listing 1: Pseudocodice Effettuazione Vendita

9.2 Operazione: Richiesta Test Drive

```
Procedura richiediTestDrive(clienteId, targa, data)
    INIZIO
        // 1. Validazione input
        SE clienteId <= 0 O targa e' vuota O data e' invalid ALLORA
            LANCIA ValidationException
        FINE SE

        // 2. Ottenere connessione
        Connection conn = DatabaseManager.getInstance().getConnection()

        // 3. Verificare che il cliente esista
        Cliente cliente = clienteDAO.getCliente(clienteId)
        SE cliente == null ALLORA
            LANCIA ValidationException("Cliente non trovato")
        FINE SE

        // 4. Verificare che il veicolo esista e sia disponibile
        String statoVeicolo = veicoloDAO.getStatoVeicolo(targa)
        SE statoVeicolo != 'disponibile' ALLORA
            LANCIA ValidationException("Veicolo non disponibile per test drive")
        FINE SE

        // 5. Registrare la richiesta di test drive
        INSERT INTO test_drive (id_cliente, targa_veicolo, data_richiesta, stato)
        VALUES (clienteId, targa, data, 'richiesto')

        // 6. Aggiornare lo stato del veicolo
        UPDATE veicoli SET stato = 'in_test_drive' WHERE targa = targa

        RITORNA successo
    ECCEZIONE
        LANCIA DatabaseException
    FINE ECCEZIONE
FINE
```

Listing 2: Pseudocodice Richiesta Test Drive

10 Scelte Progettuali e Principi SOLID

10.1 Pattern Singleton per DatabaseManager

Motivazione: Il pattern Singleton è stato scelto per garantire che esista una sola istanza del gestore della connessione durante l'esecuzione dell'applicazione. Questo previene la creazione di multiple connessioni non controllate e consente un controllo centralizzato delle interazioni con il database.

Alternativa scartata: Dependency Injection tramite Spring Framework sarebbe stata più sofisticata ma avrebbe aggiunto complessità non necessaria per un progetto didattico.

10.2 Interfacce DAO per l'Astrazione

Motivazione: Le interfacce DAO permettono di separare completamente l'implementazione concreta dell'accesso ai dati dalla logica di business. Questo facilita:

- **Testabilità**: Possibilità di creare mock object per i test unitari
- **Flessibilità**: Cambio del database (ad es. da PostgreSQL a MySQL) richiede solo di cambiare l'implementazione, non le interfacce
- **Manutenibilità**: I cambiamenti al livello di persistenza non influenzano il resto dell'applicazione

10.3 Eccezioni Personalizzate

Design: Una gerarchia di eccezioni con classe base `ConcessionarioException` facilita il handling delle eccezioni specifiche del dominio:

- `DatabaseException`: Errori di connessione e query al database
- `ValidationException`: Violazione di vincoli di validazione

Vantaggio: Permette di distinguere tra errori tecnici (database) e errori logici (validazione), migliorando la robustezza dell'applicazione.

10.4 Principi SOLID Applicati

1. **Single Responsibility**: Ogni classe ha una singola responsabilità (Veicolo gestisce solo i dati del veicolo, VeicoloDAO gestisce solo l'accesso ai dati)
2. **Open/Closed**: Le interfacce DAO sono aperte all'estensione (nuove implementazioni) e chiuse alla modifica
3. **Liskov Substitution**: Le implementazioni concrete dei DAO possono sostituire le interfacce senza alterare il comportamento
4. **Interface Segregation**: Le interfacce DAO sono specifiche per ogni entità, non generiche
5. **Dependency Inversion**: La logica di business dipende dalle astrazioni (interfacce DAO), non dalle implementazioni concrete

11 Interfaccia Utente Grafica

L'interfaccia utente è stata sviluppata utilizzando JavaFX ed è organizzata come segue:

11.1 Classe MainApp

La classe MainApp è il punto di ingresso dell'applicazione e fornisce la finestra principale con i pulsanti per accedere alle varie funzionalità:

- Registra Cliente
- Aggiungi Veicolo
- Effettua Vendita
- Richiedi Test Drive
- Visualizza Veicoli Disponibili
- Visualizza Clienti
- Lista Ordini

11.2 Viste Disponibili

L'applicazione fornisce le seguenti viste per l'interazione con l'utente:

1. **RegistraClienteView**: Finestra per la registrazione di un nuovo cliente
2. **AggiungiVeicoloView**: Finestra per l'aggiunta di un nuovo veicolo
3. **VisualizzaVeicoliView**: Finestra che visualizza i veicoli disponibili in una tabella
4. **VisualizzaClientiView**: Finestra che visualizza i clienti registrati in una tabella
5. **ModificaVeicoloView**: Finestra per la modifica dei dati di un veicolo
6. **ModificaClienteView**: Finestra per la modifica dei dati di un cliente
7. **EliminaVeicoloView**: Finestra per l'eliminazione di un veicolo
8. **EliminaClienteView**: Finestra per l'eliminazione di un cliente
9. **ModificaStatoVeicoloView**: Finestra per la modifica dello stato di un veicolo
10. **EffettuaVenditaView**: Finestra per la registrazione di una vendita
11. **TestDriveView**: Finestra per la richiesta di un test drive
12. **ListaOrdiniView**: Finestra che visualizza la lista delle transazioni effettuate

12 Diagrammi UML

L'architettura del sistema è illustrata dai seguenti diagrammi UML:

12.1 Diagramma delle Classi

Il diagramma delle classi illustra la struttura completa del sistema con tutte le classi, le loro relazioni, gli attributi e i metodi. Mostra come le interfacce DAO sono implementate dalle classi concrete, come le viste interagiscono con i DAO, e come le eccezioni si integrano nell'architettura.

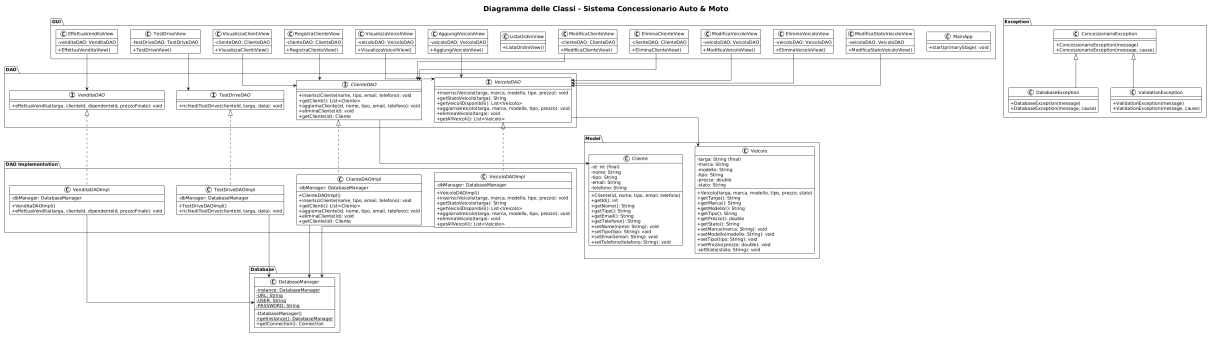

Figura 1: Diagramma delle Classi del Sistema

12.2 Diagramma del Pattern DAO

Questo diagramma illustra specificamente il pattern Data Access Object, mostrando:

- Le interfacce DAO (VeicoloDAO, ClienteDAO, VenditaDAO, TestDriveDAO)
- Le implementazioni concrete (VeicoloDAOImpl, ClienteDAOImpl, VenditaDAOImpl, TestDriveDAOImpl)
- La classe DatabaseManager che fornisce le connessioni al database
- Il collegamento al database PostgreSQL

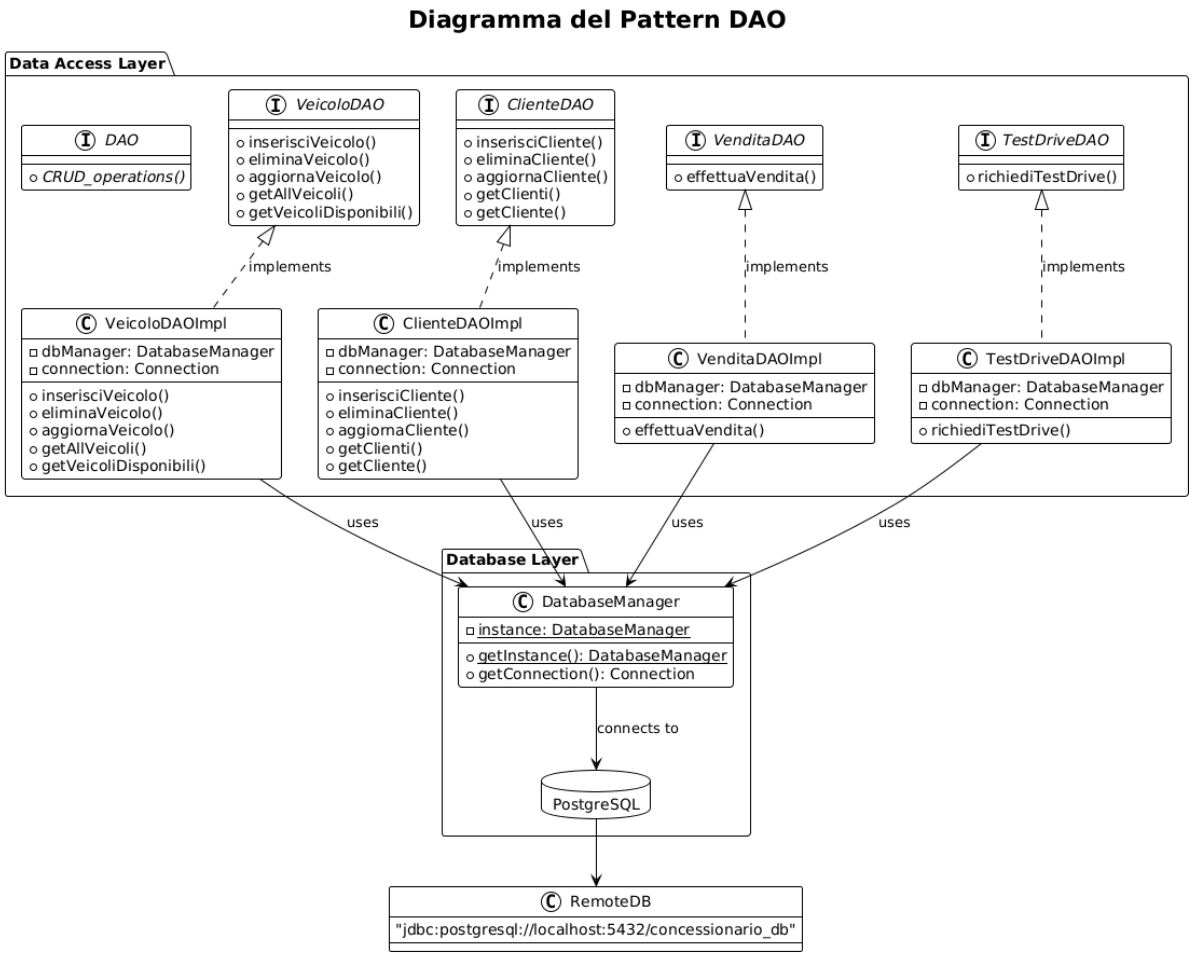

Figura 2: Diagramma del Pattern DAO

12.3 Diagramma dei Package

Il diagramma dei package illustra l'organizzazione logica del codice in package e le dipendenze tra di essi:

- **model**: Contiene le classi di dominio
- **dao**: Contiene interfacce e implementazioni per l'accesso ai dati
- **db**: Contiene il gestore del database
- **gui**: Contiene le viste JavaFX
- **exception**: Contiene le eccezioni personalizzate
- **controller**: Contiene i controller per la gestione degli eventi

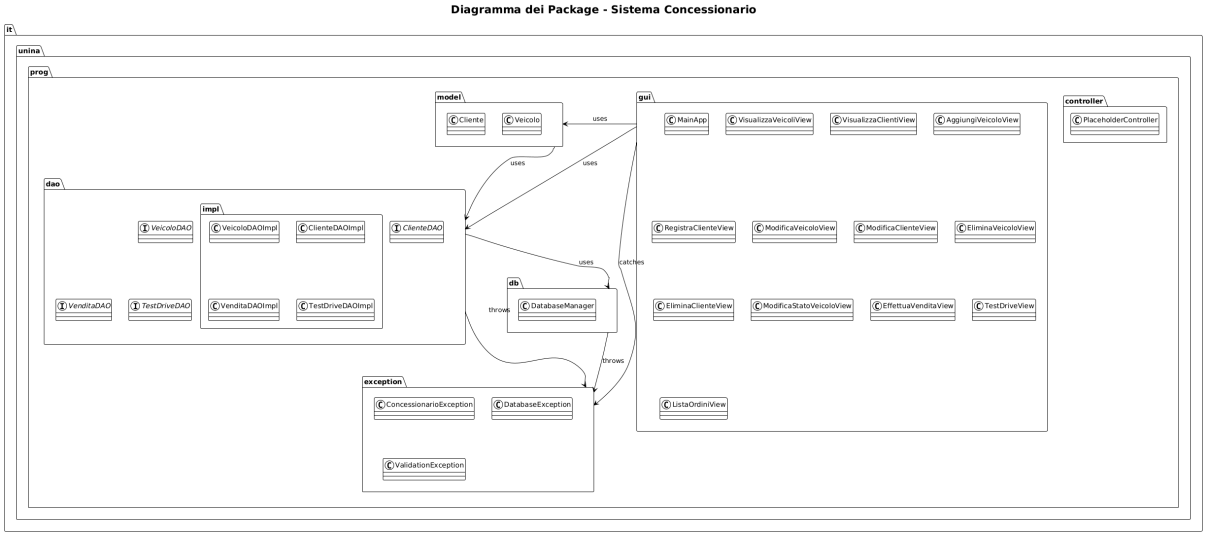

Figura 3: Diagramma dei Package

12.4 Diagramma dell'Architettura

Il diagramma dell'architettura mostra l'organizzazione a strati del sistema, dalla layer di applicazione (GUI), passando per la layer di logica di business (Model e DAO), fino alla layer di infrastruttura (Database):

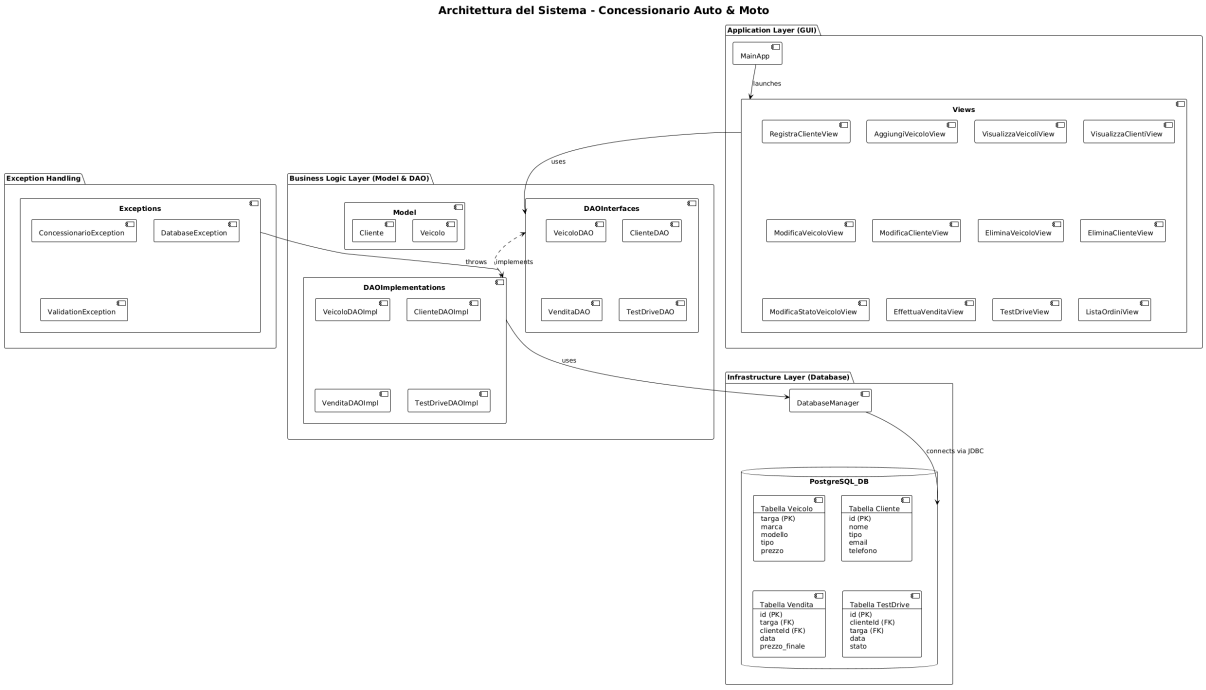

Figura 4: Diagramma dell'Architettura a Strati

13 Tecnologie Utilizzate

- **Linguaggio**: Java 17 (JDK 17)
- **Framework GUI**: JavaFX 21.0.2
- **Database**: PostgreSQL 14 o superiore
- **Driver JDBC**: PostgreSQL JDBC Driver 42.7.3
- **Build Tool**: Maven 3.8 o superiore
- **IDE Consigliato**: IntelliJ IDEA o Eclipse

13.1 Parametri di Connessione

I parametri di connessione al database sono configurati in `DatabaseManager`:

- **URL**: jdbc:postgresql://localhost:5432/concessionario_db
- **Utente**: concessionario_admin
- **Password**: password123

14 Conclusioni

Il sistema Concessionario Auto & Moto rappresenta un'applicazione completa che dimostra l'applicazione pratica dei concetti di Programmazione Object Oriented. Attraverso l'uso del pattern DAO, il sistema mantiene una separazione chiara tra la logica di accesso ai dati e la logica di presentazione, facilitando la manutenibilità e l'estensibilità del codice.

La scelta di utilizzare JavaFX per l'interfaccia utente fornisce una moderna esperienza utente desktop, mentre l'utilizzo di PostgreSQL come database backend garantisce la persistenza e l'integrità dei dati.

L'applicazione è stata progettata per essere facilmente estensibile, permettendo l'aggiunta di nuove funzionalità senza modificare il codice esistente, grazie all'adozione di principi SOLID e pattern di design riconosciuti.